Customer Churn Prediction - Assignment 2

Flask web app using the machine-learning model from Assignment 1

Student name: _____ Course: Artificial Intelligence

1. Project purpose and completion status

I made a Flask website for telecom customer churn prediction. A user enters customer and service information. The website sends it to the saved Assignment 1 model and shows a churn or stay estimate with its probability. The app is deployed on Vercel and was checked online. The assignment handout names PythonAnywhere, so the teacher must confirm if Vercel is accepted. If PythonAnywhere is required, deployment there is still needed.

2. Dataset and preparation

The supplied Telco Customer Churn CSV has 7,043 rows and 21 columns: 19 customer features, customerID, and the Churn target. It has 5,174 No Churn rows and 1,869 Churn rows. There are 11 blank TotalCharges values, no duplicate rows, and all customer IDs are unique. The notebook converts blank TotalCharges values to missing numbers, keeps those customers, and removes customerID because it is not a prediction feature.

All 19 model features. Customer: gender, SeniorCitizen, Partner, Dependents. Tenure: tenure. Services: PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies. Contract: Contract, PaperlessBilling. Billing: PaymentMethod, MonthlyCharges, TotalCharges.

3. Model training and measured results

The notebook uses a stratified 80/20 train/test split with random_state 42. The selected classifier is Logistic Regression with C = 30. Model selection and tuning use the training set and stratified five-fold cross-validation. Numeric values use median imputation and standard scaling. Category values use most-frequent imputation and one-hot encoding. These steps are inside the fitted scikit-learn pipeline, so training and website inputs use the same preparation.

The churn probability threshold is 0.55. It was selected from out-of-fold predictions on training data. The holdout test set was kept aside for final evaluation.

Measure	Result
Five-fold cross-validation accuracy	80.37%
Holdout accuracy	79.77%
Churn precision / recall / F1	67.05% / 46.79% / 55.12%
ROC AUC	0.8409
Majority-class baseline accuracy	73.46%

The requested accuracy above 85% was not reached. Holdout accuracy is 79.77%. It is above the 73.46% majority-class baseline but below the target. These are the measured results.

Holdout confusion matrix. Rows are actual No Churn and Churn. Columns are predicted No Churn and Churn.

Actual / predicted	No Churn	Churn
No Churn	949	86
Churn	199	175

4. Saved model and project files

model.joblib contains the fitted preprocessing and Logistic Regression pipeline, model name, threshold, and ordered feature names. At startup, app.py checks that the model feature order matches the form. This keeps the website connected to the real model.

app.py	Flask routes and input checks
model.joblib	trained pipeline, preprocessing, threshold
requirements.txt	Python packages
.python-version	Vercel Python version (3.14)
churn_prediction.ipynb	data, training, evaluation, export
WA_Fn-UseC_-Telco-Customer-Churn (1).csv	source data
templates/index.html	input form and prediction display
static/style.css	local CSS
public/static/style.css	CSS served by Vercel
tests/test_app.py	seven Flask tests

5. Input-to-prediction flow

1. GET / displays the form. 2. The user enters 19 customer features. 3. POST /predict checks required values, categories, number format, and numeric limits. 4. Flask arranges values in FIELD_ORDER and creates a one-row pandas DataFrame. 5. The saved pipeline imputes, scales, and encodes the values; Logistic Regression calculates churn probability. 6. Flask compares it with 0.55 and shows “Likely to churn” or “Likely to stay” with the probability.

```
input_frame = pd.DataFrame(
    [[customer[name] for name in FIELD_ORDER]],
    columns=FIELD_ORDER,
)
churn_probability = float(MODEL.predict_proba(input_frame)[0, 1])
is_churn = churn_probability >= THRESHOLD
```

The output is calculated from each submitted customer. It is not hard-coded or a static HTML result.

6. Validation and tests

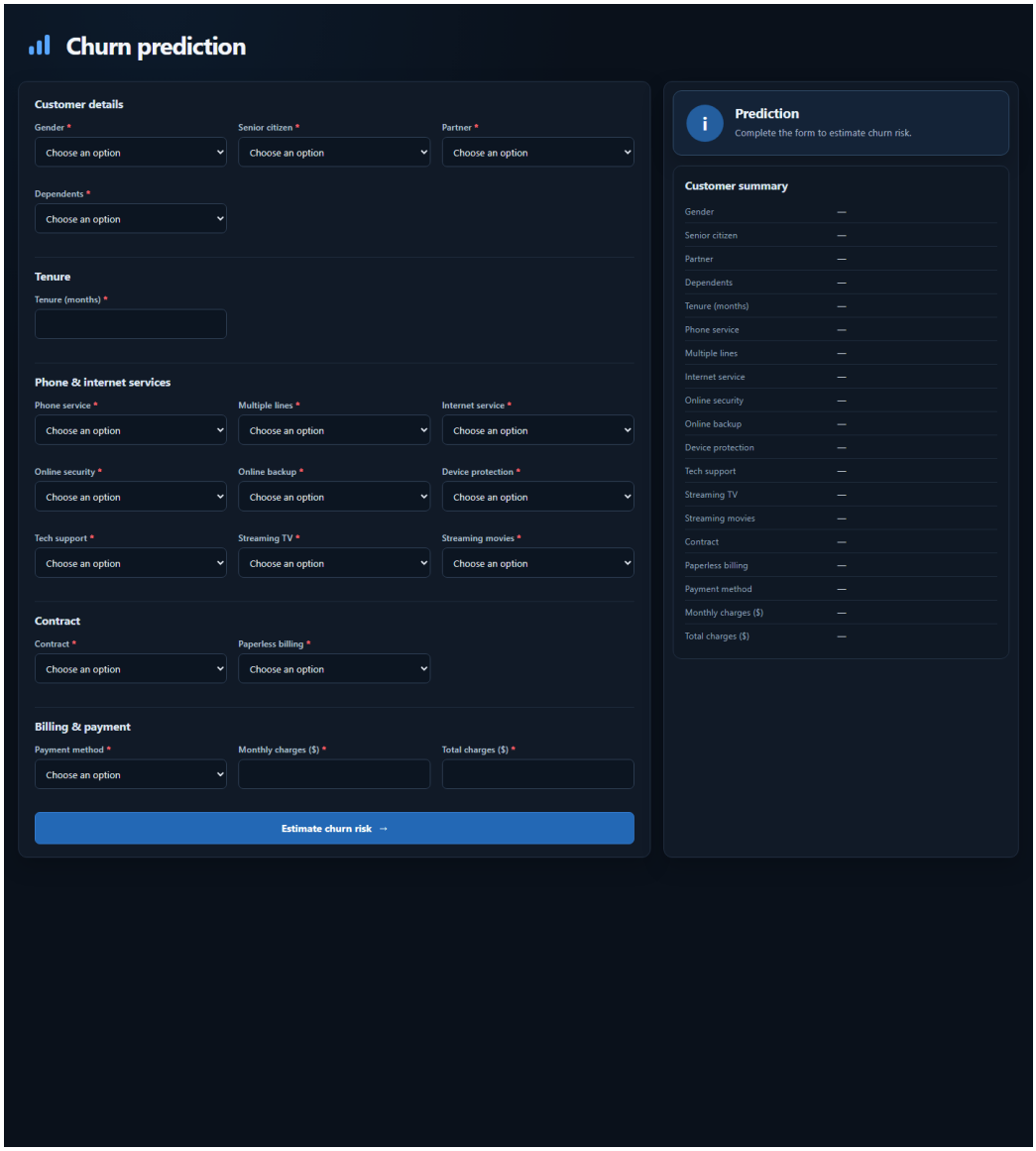
The webpage marks every field as required. Flask checks again on the server. Empty or non-finite numbers, out-of-range values, fractional values for integer fields, and unknown choices are rejected with a message.

Test	Result
Homepage and form	Passed
Churn customer example	Passed: 92.7%
Stay customer example	Passed: 0.5%
Missing value	Passed: rejected
Out-of-range number	Passed: rejected
Unexpected category	Passed: rejected
Valid phone-service choice	Passed

All 7 Flask tests pass. The live page and stylesheet return HTTP 200. The churn and stay examples were also submitted on the live site.

7. Final website and results

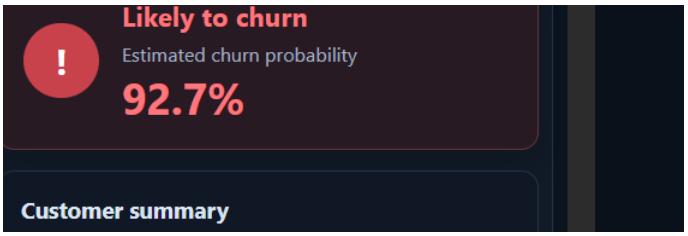
The screenshot shows the final dark form, customer summary, and neutral prediction panel before submission. The red asterisks mark required fields.



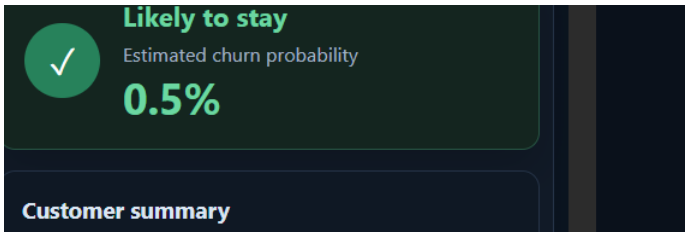
Final deployed form and customer summary.

Two live example predictions are shown below. The first customer has a 92.7% estimated churn probability. The second has a 0.5% estimated churn probability. These are estimates, not guarantees.

Likely to churn - 92.7%



Likely to stay - 0.5%



8. Run and deploy

To run locally, install packages from requirements.txt and start Flask. Keep the terminal open, then open the local address in a browser.

```
.\.venv\Scripts\python.exe app.py
```

The production app runs on Vercel as a Python Flask function. .python-version selects Python 3.14. The stylesheet is also in public/static so Vercel can serve it. The production website opened without a sign-in, and both example predictions worked.

The assignment specifically asks for a PythonAnywhere URL. This version is on Vercel, so the teacher must confirm if that platform is accepted. If PythonAnywhere is required, deploy the same Flask files and model there, configure WSGI, and test the public site.

9. What I learned and remaining evidence

I learned that the website must use the same data preparation and feature order as model training. Server-side checks are also important. The model result is only an estimate, and the measured accuracy is below the requested 85%.

This report includes dataset facts, model results, the project structure, the full input-to-prediction explanation, the final website screenshot, and both prediction screenshots. I did not capture screenshots of the model-export cell, project folder in the file explorer, code in the editor, local terminal run, or PythonAnywhere settings. I have not presented those as screenshots.

Live Vercel website: https://churn-prediction-app-three.vercel.app/